MOBL-11: Dashboards & Alerting

> **Series:** MOBL — Mobile Monitoring | **Notebook:** 11 of 12 | **Created:** February 2026 | **Last Updated:** 11/11/2026

Overview

Effective mobile monitoring requires more than raw telemetry -- you need dashboards that surface the right KPIs at a glance and alerts that notify the right people when something goes wrong. This notebook covers designing mobile KPI dashboards with tiles for crash-free rate, session volume, and performance trends; setting up crash rate monitoring with timeseries visualizations; tracking app performance metrics across user actions; and configuring alerts using Dynatrace Intelligence anomaly detection and metric events. The goal is to move from reactive troubleshooting to proactive mobile observability.

Table of Contents

1. Mobile KPI Dashboard Design
2. Crash Rate Monitoring
3. App Performance Metrics
4. Session Volume Trends
5. Metric Alerts for Mobile
6. Detected Problem Correlation
7. Executive Summary Tiles

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`storage:user.events:read`, `storage:user.sessions:read`, `storage:events:read` (problems)
Dashboard Permissions	`document:documents:write` for creating and editing dashboards
Workflow Permissions	`automation:workflows:write` for configuring alert-triggered workflows
Mobile App	At least one mobile application with active sessions and crash data
Prior Knowledge	Familiarity with MOBL-01 through MOBL-10 recommended

1. Mobile KPI Dashboard Design

A well-designed mobile dashboard answers the question "Is our mobile app healthy right now?" at a glance. Rather than cramming every metric onto a single screen, organize tiles by purpose: health indicators at the top, trends in the middle, and drill-down tables at the bottom.



Recommended Dashboard Layout

Tile	Type	Purpose
Crash-Free Rate	Single value	Overall app health indicator -- percentage of sessions without crashes
Session Volume	Timeseries	Usage trends over time to spot adoption changes or drops
Top Crashes	Table	Prioritize fixes by crash frequency and affected user count
Action Duration	Timeseries	Performance monitoring for key user actions (load, tap, swipe)
OS Distribution	Pie/Donut	Platform breakdown to allocate testing and development resources
Error Rate	Timeseries	Error monitoring to catch regressions before they become crashes

Design Principles

- **Top row: health at a glance** -- Use single-value tiles with color thresholds (green/yellow/red) for crash-free rate, active sessions, and error rate
- **Middle row: trends** -- Timeseries charts for session volume, action duration, and error trends over the last 7 days
- **Bottom row: drill-down** -- Tables for top crashes, slowest user actions, and most affected app versions
- **Use variables** -- Add a dashboard variable for `frontend.name` so stakeholders can filter to their specific app
- **Time range selector** -- Always include a time range control defaulting to the last 24 hours, with presets for 1h, 6h, 24h, 7d

2. Crash Rate Monitoring

Crash rate is the most critical mobile KPI. Google Play states that its Android vitals thresholds affect visibility — *"To maximize your title's visibility on Google Play, please keep it under these thresholds"* (user-perceived crash rate 1.09%, ANR rate 0.47%; MOBL-10 §8 explains why those are per-user, not per-session, rates).

> **Sources:** Android vitals (Android Developers).

The following query builds a daily timeseries of mobile sessions and of sessions that contained a crash, read from `user.sessions` (one record per session, with `error.has_crash`). Dividing the two gives a session crash rate.

```
// Crash rate timeseries (session grain): sessions vs. sessions with a crash
fetch user.sessions, from:-7d
| filter dt.rum.application.type == "mobile"
| makeTimeseries {total_sessions = count(), crash_sessions =
countIf(error.has_crash == true)}, time:start_time, interval:24h
```

Interpreting the Results

This query produces two timeseries arrays: `total_sessions` (all mobile sessions) and `crash_sessions` (sessions with `error.has_crash`). To calculate the crash rate percentage on a dashboard tile, divide crash sessions by total sessions and multiply by 100; the crash-free rate is 100 minus that. The bands below are a community rule of thumb — set your own targets from your app's baseline:

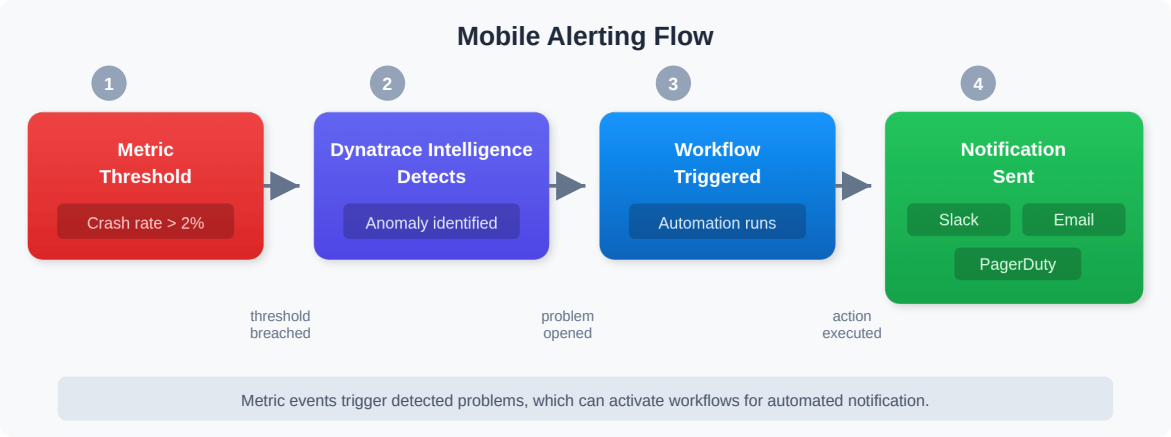
Crash-Free Rate	Health Status	Action
> 99.5%	Excellent	Monitor normally
99.0% - 99.5%	Acceptable	Review top crash groups
98.0% - 99.0%	Degraded	Investigate and prioritize fixes
< 98.0%	Critical	Immediate action required

> **Tip:** Break crash rate down by app version using `by:{app.short_version}` to identify whether a specific release introduced a regression (MOBL-10 §8 has the per-version query).

3. App Performance Metrics

Beyond crashes, performance directly impacts user engagement. Slow app launches, sluggish screen transitions, and unresponsive taps all contribute to poor user experience and eventual churn. Track these metrics alongside crash data to get a complete picture of mobile app health.

The alerting flow for mobile performance follows a standard pattern from metric detection through notification:



Key Performance Indicators

Starting bands from community practice — tune them to your app and audience:

Metric	Good	Warning	Critical
App Launch Time	< 2 seconds	2-5 seconds	> 5 seconds
Action Duration	< 1 second	1-3 seconds	> 3 seconds
HTTP Error Rate	< 1%	1-5%	> 5%
Crash-Free Rate	> 99.5%	98-99.5%	< 98%

User Action Duration

User action duration measures how long it takes for specific interactions to complete -- tapping a button, loading a screen, or submitting a form. Tracking these durations over time reveals performance regressions introduced by new releases or backend changes.

```
// User action duration trend by interaction type
fetch user.events, from:-24h
| filter in(dt.rum.agent.type, {"android", "ios"}) and
characteristics.has_user_action
| makeTimeseries avg_duration = avg(duration), by:{interaction.type},
interval:1h
```

Reading the Duration Chart

The timeseries above breaks down average action duration by `interaction.type`. Look for:

- **Gradual increases** -- May indicate backend degradation or growing payload sizes
- **Sudden spikes** -- Often correlated with a new app release or backend deployment
- **Platform differences** -- Compare iOS vs Android by adding `os.name` to the `by:` list to identify platform-specific bottlenecks

> **Note:** `duration` is a duration value, not a number of milliseconds. Compare it with a duration literal (`duration > 2s`), and convert explicitly (`duration / 1ms`) only when a tile needs a plain number.

4. Session Volume Trends

Session volume is the simplest usage signal: how many people are using the app, and when. Sudden drops often point to an outage, a broken release, or a failed store rollout rather than to changing user behaviour.

```
// Session volume timeseries (hourly)
fetch user.sessions, from:-7d
| filter dt.rum.application.type == "mobile"
| makeTimeseries session_count = count(), time:start_time, interval:1h
```

This query counts mobile sessions per hour (one `user.sessions` record per session) over the past 7 days. Use it as a dashboard tile to spot usage patterns (peak hours, weekday vs weekend) and detect sudden drops that may indicate an outage or broken update.

5. Metric Alerts for Mobile

Dashboards are for humans looking at screens. Alerts are for ensuring problems are noticed even when nobody is watching. Dynatrace supports two complementary alerting mechanisms for mobile KPIs:

1. **Dynatrace Intelligence Anomaly Detection** -- Automatically detects deviations from baseline behavior without manual threshold configuration. Best for metrics with natural variance (session volume, action duration).
2. **Metric Events (Custom Alerts)** -- Manually defined thresholds that fire when a metric crosses a specific boundary. Best for hard limits (crash rate > X, error rate > Y).

Recommended Alert Configuration

Alert	Metric/Query	Threshold	Severity
High Crash Rate	Crash count per hour	> 10 crashes in a sliding 1-hour window	Critical
Session Drop	Session count drop	< 50% of the rolling 7-day baseline	Warning
Slow App Launch	App start duration	> 5 seconds average over 15 minutes	Warning
High Error Rate	HTTP 5xx from mobile	> 5% of requests returning server errors	Critical

Creating a Crash-Rate Detector (Davis anomaly detector)

Build new mobile alerting as a **Davis anomaly detector** in the Anomaly Detection app, with a DQL query as its source and a static-threshold analyzer. A source query for crash volume per app:

```
fetch user.events, from:-2h
| filter characteristics.has_crash
| makeTimeseries crashes = count(), by:{frontend.name}, interval:5m
```

Set the static threshold (for example, above 10 crashes per hour, expressed at the 5-minute interval you choose) and the event template in the detector. Confirm in the detector preview that the query is accepted as a source before relying on it. AIOPS-02 §4 walks

through the analyzer, tuning and event-template settings, and ALERT-02 covers choosing between mechanisms.

> **Where the detector is created (SaaS 1.344).** *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings. Because Anomaly Detection is deprecated, we highly recommend that you use Settings to access your existing configurations and create new ones."* SaaS 1.344 rolls out to tenants in stages — check your tenant's version before following either path. On earlier versions the **Anomaly Detection** app is where custom alerts are created, and the steps above describe that path.
>
> **Sources:** Anomaly Detection (DT docs) — *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings."*

Classic path: metric events

The classic surface is **Settings** > **Anomaly Detection** > **Metric Events**. `builtin:anomaly-detection.metric-events` is flagged **Blocked at upgrade**, so metric events built there have to be recreated as DQL-based detectors (`builtin:davis.anomaly-detectors`) when the tenant moves to the latest Dynatrace; existing metric events keep working until then. Use it only while your tenant is still on the classic surface.

> **Correction (09/28/2026).** Earlier revisions showed a metric-event example on a metric key `dt.rum.mobile.crash.count` . No such key is documented, and none appears in the metric catalog of the validation tenant (`metrics | filter startsWith(metric.key, "dt.rum.mobile")` returns nothing, 09/28/2026), so the example could not be built. It has been replaced by the detector above.

Dynatrace Intelligence vs Static Thresholds

Approach	Best For	Limitations
Dynatrace Intelligence	Metrics with seasonal patterns, automatically adapts to baselines	May miss gradual degradation; requires learning period
Static Threshold	Hard business limits (SLAs, crash budgets)	Must be manually tuned; doesn't adapt to growth

Approach	Best For	Limitations
Both Combined	Maximum coverage -- Dynatrace Intelligence catches anomalies, static thresholds enforce SLAs	More alerts to manage

> **Tip:** Start with Dynatrace Intelligence for performance metrics (it adapts to your app's normal patterns) and add static thresholds only for KPIs with hard business requirements like crash-free rate SLAs.

6. Detected Problem Correlation

When Dynatrace Intelligence detects an anomaly affecting a mobile application, it creates a problem that can be correlated with the mobile telemetry you see on dashboards. The following query retrieves recent detected problems that impact mobile device applications, helping you bridge the gap between infrastructure-level detection and user-facing impact.

```
// detected problems affecting mobile applications
// smartscape.affected_entities replaces the deprecated affected_entity_ids;
it holds FRONTEND
// nodes for web and mobile alike, so the lookup keeps only frontends with
frontend.type == "mobile"
fetch dt.davis.problems, from:-7d
| expand affected = smartscape.affected_entities
| filter affected[type] == "FRONTEND"
| fieldsAdd frontend_id = affected[id]
| lookup [smartscapeNodes "FRONTEND" | filter frontend.type == "mobile" |
fields id, name], sourceField:frontend_id, lookupField:id, prefix:"app."
| filter isNotNull(app.id)
| fields timestamp, display_id, event.name, event.status, mobile_app =
app.name
| sort timestamp desc
| limit 20
```

Correlating Problems with Mobile Metrics

When you identify a detected problem affecting a mobile application, overlay the problem time range on your dashboard timeseries to see:

- **Did crash rate spike during the problem window?** -- If yes, the root cause likely caused crashes, not just slowdowns
- **Did session volume drop?** -- A sudden drop in sessions during a problem may indicate users are unable to launch the app
- **Did action duration increase?** -- Performance degradation detected by Dynatrace Intelligence should correlate with user-facing slowness

Problem Workflow Integration

Connect detected problems to your notification channels so the mobile team is alerted immediately. Create a workflow in the workflow editor with the **Problem trigger** and these settings:

Trigger setting	Value
Trigger	Problem trigger
Problem state	`active` (starts when the problem opens)
Affected entities	**Include entities with any defined tag below**: `app-type:mobile` (requires that tag on your mobile app entities, MOBL-99 §8 #10)
or Additional custom filter query	`matchesValue(affected_entity_ids, "MOBILE_APPLICATION-*")` (deprecated field — see the note below)
Advanced options	Enable **Wait for root cause analysis** to avoid triggering on incomplete problem data

Then add a Slack **Send message** task. The message template below uses only fields of the problem record. The action ID is the one Dynatrace's workflow samples use; in an exported workflow, `connection` holds the Slack connection's ID rather than a name (see WFLOW-03 §1).

```
tasks:
  - name: notify_mobile_team
    action: dynatrace.slack:slack-send-message
    input:
      connection: slack-mobile-alerts
      channel: "#mobile-incidents"
      message: |
        :rotating_light: Mobile Problem Detected
        Problem: {{ event()['display_id'] }} — {{ event()['event.name'] }}
        Category: {{ event()['event.category'] }}
        Affected: {{ (event().get('smartscape.affected_entities') or []) |
map(attribute='name') | join(', ') }}
        Link: {{ problem_link() }}
```

Deprecated problem fields (updated 11/11/2026). The alert-notification upgrade guide states: *"root_cause_entity_id and root_cause_entity_name are deprecated in favor of root_cause.smartscape_entity, and affected_entity_ids in favor of smartscape.affected_entities. Build new payloads against the Smartscape fields."* The query above and the template's *Affected* line read `smartscape.affected_entities`, a record array whose elements carry `id`, `type` and `name`. The custom-filter row still shows the classic `affected_entity_ids` matcher, which works while the deprecated field exists, but it has no one-line Smartscape equivalent: mobile and web apps are both `FRONTEND` nodes, and the problem record does not say which kind an affected frontend is. Prefer the tag filter for new triggers.

Sources: Upgrade guide — alerting and notifications (DT docs) — the quote above, and *"Record array; each element has id, type, name"*, re-read 11/11/2026. Query executed 11/11/2026: no problem on the validation tenant affected a `FRONTEND` node; the same shape on `SERVICE` nodes returned rows.

Correction (09/28/2026). Earlier revisions showed the trigger as a `trigger: type: davis-problem / entityTagsMatch` YAML block. No editor, export or API accepts that shape. The Problem trigger is configured through the settings above.

Sources: Event triggers for workflows (DT docs) — *"Additional custom filter query : Add a DQL matcher expression to further refine which problems start the trigger."*; threat-detection-notification-sender.yaml (Dynatrace GitHub) — *"action: dynatrace.slack:slack-send-message"*.

Template fields come from the problem record. The Problem trigger's `event()` is the `dt.davis.problems` record — run `fetch dt.davis.problems, from:-24h |`

> `limit 1` to see every field a template can read. It has no `title` field, and its `severity` field is not a CRITICAL/HIGH label (0 of 3,209 problem records on a validation tenant, 09/24/2026): the title is `event.name`, the kind of problem is `event.category`, and the link is `{{ problem_link() }}`, which *"evaluates correctly in workflows with Davis problem event triggers only."* (Jinja expressions for Workflows (DT docs), Event triggers for workflows (DT docs))

7. Executive Summary Tiles

Executive stakeholders need a single table that answers: "How are our mobile apps doing?" The following query produces a summary row per application with the key metrics that matter most -- total actions, unique sessions, crash count, and actions per session (an engagement indicator).

```
// Executive summary -- key metrics per mobile app
fetch user.events, from:-24h
| filter in(dt.rum.agent.type, {"android", "ios"})
| summarize {total_actions = countIf(characteristics.has_user_action == true),
unique_sessions = countDistinct(dt.rum.session.id), crash_count =
countIf(characteristics.has_crash == true)}, by:{frontend.name}
| fieldsAdd actions_per_session = toDouble(total_actions) /
toDouble(unique_sessions)
| sort total_actions desc
| limit 10
```

Building the Executive Dashboard

Use the query above as a table tile on your dashboard. Add conditional formatting to highlight:

- **Crash count > 0** in red to draw attention to apps with active stability issues
- **Actions per session < 3** in yellow to flag apps with low user engagement
- **Unique sessions** trending down week-over-week as a leading indicator of user attrition

Dashboard Sharing

Audience	Dashboard Focus	Refresh Interval
Mobile Developers	Crash details, stack traces, action performance	5 minutes
QA Team	Crash-free rate by version, error trends	15 minutes
Product Managers	Session volume, engagement, feature adoption	1 hour
Executives	Summary KPIs, crash-free rate, active users	Daily snapshot

> **Tip:** Use Dynatrace dashboard sharing to send scheduled PDF snapshots to stakeholders who do not log into Dynatrace directly.

Summary

In this notebook, you learned:

- **Dashboard design principles** -- Organize tiles by purpose (health indicators at top, trends in middle, drill-down tables at bottom) with application-level filtering
- **Crash rate monitoring** -- Build a session-grain timeseries comparing all sessions to crashed sessions (`user.sessions` , `error.has_crash`), and interpret crash-free rate thresholds
- **App performance metrics** -- Track user action duration to detect regressions and correlate with backend changes
- **Session volume trends** -- Identify usage patterns, peak hours, and sudden drops that may indicate outages
- **Alerts** -- Build a Davis anomaly detector on a DQL crash query for hard limits, with metric events only as the classic path
- **detected problem correlation** -- Query problems affecting mobile applications and overlay them with dashboard timeseries
- **Executive summary tiles** -- Build single-table summaries with total actions, unique sessions, crash count, and engagement metrics

---

Next Steps

Continue to **MOBL-12: Advanced Instrumentation & Optimization** to explore:
- Business events from mobile apps with `sendBizEvent`
- Custom errors, events and request tagging with the OneAgent SDK
- Feature-flag tracking and data-volume optimization

References

- Dashboards and notebooks (DT docs)
- Anomaly detection metric events (DT docs)
- Mobile App Monitoring
- Davis Problems app (DT docs)
- Workflows (DT docs)
- Event triggers for workflows (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.